

Git for ML projects: commits, branches, remotes

One folder with its full history: every change saved with a reason, every experiment on its own branch.

THE QUESTION

How do you keep every version of an ML project, try a risky idea without breaking the working code, and combine your work with a teammate's?

THE INTUITION

Git stores snapshots of the whole project as commits, each pointing to its parent. The staging area lets you choose what goes into the next snapshot. A branch is a movable label on a commit, so an experiment costs nothing and merges back if it works.

KEY TAKEAWAY

Run status, diff, add, commit, with a message that finishes "If applied, this commit will...". Use a branch per experiment, pull before you push, and keep secrets, data and models out.

edit → **git add** → **git commit** → **branch per experiment** → **git merge** → **git push**

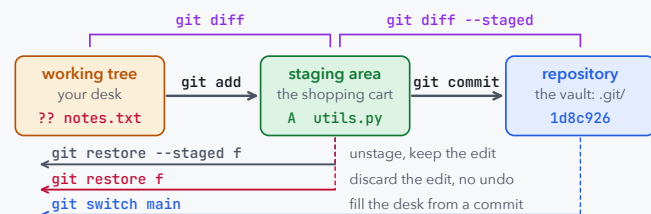
1 Copies are not versions

the pain	what git gives you	command
train_v2_FINAL_fixed.py	one file, every version kept	git log
I broke everything	go back to a saved commit	git restore
try a risky idea	a branch; main stays safe	git switch -c
combine Alice's copy	merge, then push and pull	git merge

```
git init      # makes the hidden .git/ folder
git config --global user.name "Your Name"
```

.git/ is the repository: delete it and the history is gone. Git works offline; GitHub only hosts copies. If the first branch is called master, git branch -M main renames it.

2 Stage with add, save with commit



git status --short marks each file: ?? untracked, A staged and new, M modified. Stage utils.py alone and the commit holds utils.py while notes.txt waits on the desk.

3 The daily rhythm: status, diff, add, commit

git diff after adding evaluate() to train.py

```
@@ -6,4 +6,8 @@ def train(X, y):
    model.fit(X, y)
    return model

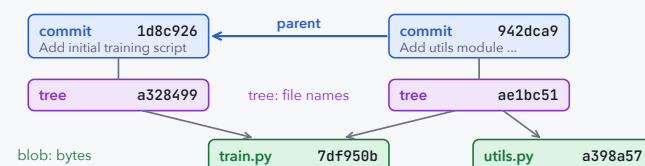
- print("Training model...")
+ def evaluate(model, X, y):
+     acc = model.score(X, y)
+     print(f"Accuracy: {acc:.3f}")
+
+ print("Training and evaluating model...")
```

— hunk: old lines 6 to 9, new lines 6 to 13
— space: unchanged context
— - removed line
— + added line
git diff --staged: what the next commit will hold

```
git status      # what changed?
git diff        # exactly how?
git add train.py # stage it
git commit -m "Add evaluation function"
```

A message finishes "If applied, this commit will...": "Add evaluation function", never "fix" or "stuff". Commit early and often.

4 A commit is a snapshot with a parent



after commit 1: 3 objects in .git/objects (commit, tree, blob)
after commit 2: 6; the unchanged train.py blob is reused

Each object is named by the SHA-1 of its bytes: 40 hex characters, usually shown as the first 7. Same bytes, same id, so an unchanged file is stored once. HEAD is where you are; HEAD~1 is its parent.

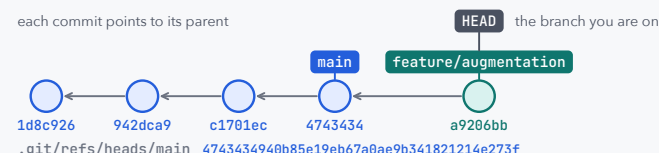
5 Look back first, then undo gently

command	what it does
git restore f	throws away unstaged edits: no undo
git restore --staged f	unstages; the edit stays in the file
git reset --soft HEAD~1	undoes the last commit, keeps it staged
git reset --hard HEAD~1	deletes the commit and its changes
git revert <hash>	adds a commit that undoes one: safe after push
git stash / stash pop	parks unfinished work, then brings it back

```
git log --oneline      # history
git show HEAD~3:train.py # an old file
git diff HEAD~3 HEAD -- train.py
git log -S "normalize"  # code search
git blame train.py      # line authors
```

reset rewrites history, so use it only on commits you have not pushed; revert is for pushed ones.

6 A branch is a label on a commit



```
git switch -c feature/augmentation
# older spelling: git checkout -b ...
git switch main # augment.py vanishes
git branch -d feature/augmentation
```

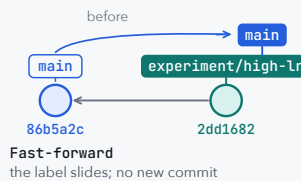
A new branch is one 40-character id in a file, so it is instant and free. Switching swaps the files on your desk while main stays safe.

Merge, resolve, share, and keep the junk out

Branches meet again in a merge, a remote shares the history, and .gitignore keeps secrets, data and models out of it.

7 Fast-forward, or a merge commit

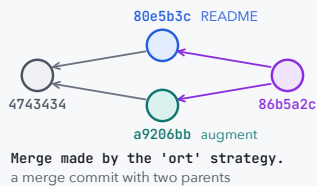
fast-forward
main had not moved



On main, `git merge feature/augmentation` joins the work; different files or different parts of one file merge on their own.

three-way merge

both branches moved on



8 Both sides changed a line: you decide

config.yaml after `git merge experiment/low-lr`

```
model:
  type: logistic_regression
<<<<<< HEAD
  max_iter: 2000
  learning_rate: 0.01
=====
  max_iter: 500
  learning_rate: 0.001
>>>>>> experiment/low-lr
data:
```

— your side starts (HEAD = main)
+ ours: main, after merging high-lr
— end of ours, start of theirs
+ theirs: experiment/low-lr
— end of theirs

`git status --short: UU config.yaml`

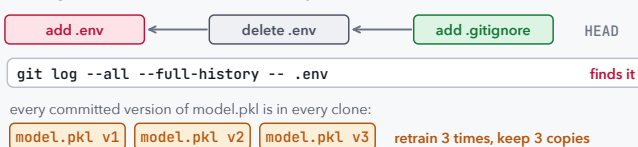
Keep one side or combine them, delete all three marker lines, then:

```
git add config.yaml
git commit -m "Resolve lr conflict"
git merge --abort # or give up
```

`git checkout --ours f` or `--theirs f` takes one whole side.

10 Secrets and big files stay in history

deleting a file does not remove it from history



A pushed key is public: rotate it first, then `git rm --cached .env`, ignore it and commit. Keep datasets and weights in DVC, Git LFS or cloud storage; a pre-commit hook, `check-added-large-files`, blocks big files.

12 Pause and think

! [rejected] ... (fetch first)?
Someone pushed first: pull, fix any conflict, push.

divergent branches on pull? Both sides moved on:
`git pull --no-rebase merges`.

detached HEAD? You checked out a commit: `git switch main`, or `git switch -c try-x` to keep work.

Committed .env last week? Rotate the key; deleting the file leaves it in history.

would be overwritten by checkout?
`git stash`, `switch`, `return`, `git stash pop`.

.gitignore ignored? The file is tracked already: `git rm --cached f`, commit.

9 Write .gitignore before the first commit

`git status --short` in a busy ML folder

```
?? .DS_Store      OS junk
?? .env           secrets: API keys
?? __pycache__/.  bytecode, rebuilt
?? data/          datasets: too big
?? logs/          run logs
?? model.pkl      a trained model
```

after adding .gitignore

```
?? .gitignore
commit it; git check-ignore
-v file names the rule
that hides a file
```

```
# bytecode
__pycache__/.
*.pyc
# secrets
.env
*.secret
```

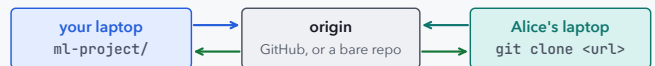
```
# models
*.pkl
*.h5
*.pt
# data
data/*.csv
```

```
# run logs
logs/
wandb/
mlruns/
# editor, OS
.DS_Store
```

A # starts a comment only at the start of a line.
.gitignore hides only untracked files; for a tracked one, `git rm --cached f` and commit.

11 Remotes and pull requests

push: up pull = fetch + merge: down



on a team, main changes only through a pull request:



```
git remote add origin <url>
git push -u origin main # first push
git pull                # fetch + merge
git fetch                # download only
```

.github/workflows/ci.yml, shortened

```
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements.txt
      - run: pip install pytest
      - run: pytest tests/ -v
```

Pull before you push. Protected main takes a pull request only when CI passes and a teammate approves.

status before add · one idea per commit · a branch per experiment · pull before push · no secrets, no big files

Source: lecture "Git deep dive", its follow-along script and guide, the git internals demo and the CI/CD lecture, nipunbatra.github.io/stt-ai-teaching. Commit and blob ids are real: make.py computes them from the follow-along's files the way Git does, and a throwaway repository (Git 2.53) built from the same files gave the same ids. Status lines, conflict markers and push and pull messages are copied from that run. The reproducible runs sheet pins the environment that goes with a commit.